

Fig. 6: Microchip Application Maestro screenshot

tains names of all the library routines. Before compiling the program, some delay subroutines need to be written in the xlcd.c files in order to add delays required for the working of the LCD.

The program works as per the flow-chart shown in Fig. 7.

In the program, header files for 'timers,' 'delays,' 'lcd' and 'math' have been used. First, configure Port A pins as digital input. The LCD is initialised for 4-bit mode, display on, cursor on, blink off, etc. The first operand is displayed in the first line of the LCD with the help of the function 'input_number.' The word 'operator' displayed in the second line of the LCD means that the program is waiting for an operator from the user. Pressing an operator switch displays the operator on the LCD. At this time, no switch other than the operator switch will work.

Operator switches are time-multiplexed. That means an operator switch allows you to perform more than one function. When an operator switch is pressed once, it displays the first operator function. Pressing it again soon after displays another operator at the same position of the LCD. A switch can have two-three operators.

After you select the desired operator, leave the keypad. After a small period of time, the program will understand which operator has been selected. Flag variable 'f' is used to find out the number of times a switch is pressed. Its initial value is 0. When an operator key is pressed, the flag variable's value changes from 0 to 1. If the same switch is pressed again, the program comes to know through the flag variable that the same switch is pressed for the second time and changes the flag variable value from 1 to 2, depending on the number of operators related to that switch. Now the LCD shows the second operator function of the switch.

A number is given to each operator to understand which operation is selected or which operation has to be performed. This number is stored in variable 'ope.' For example, '+' has 'ope' value of 1, '-' has 'ope' value of 2, 'log' has 'ope' value of 10 and so on. When the program accepts the operator, the word 'operator' in the second line of the LCD is erased.

Now the program checks whether the given mathematical operator requires another operand or not. If it requires another operand, the above process repeats. The only difference is that now the second line of the LCD, not the first line, is used for inputting the data.

After the operand and the operator are entered, the program takes some

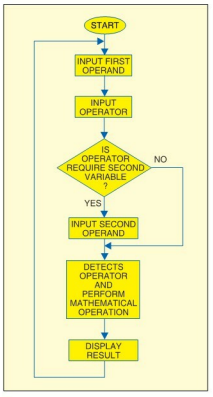


Fig. 7: Flow-chart of working of scientific calculator

time to read the data from the LCD and then perform mathematical operation on it. It checks the value of 'ope' to know which mathematical operation has to be performed. After performing the mathematical operation, it shows letter 'E' on the right side of the second line of the LCD. Now if you press the execution key, the result is shown in the second line of the LCD. Pressing the execution key before letter 'E' is displayed in the LCD will not give any result.

If 'AC' key is pressed while executing the code, the LCD gets cleared and the program starts executing from the beginning.

After letter 'E' is shown on the LCD, only 'EXC' and 'AC' keys work.

Keyboard subroutine. This subroutine uses the keys debounce technique and works as per the flow-chart shown in Fig. 8. When the keyboard function is called, the program waits for any key to be pressed and returns to its position (column and row values). The value of the row and the column